

Code Assessment of the MetaDEX Slipstream Smart Contracts

August 29, 2026

Produced for



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	6
4	Trust Model	9
5	System Considerations	12
6	Terminology	14
7	Findings	15
8	Open Questions	30
9	Limitations and use of report	31



1 Executive Summary

Dear Velodrome Team,

Thank you for trusting us to help Velodrome with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of MetaDEX Slipstream according to [Scope](#) to support you in forming an opinion on their security risks.

Velodrome implements Slipstream, a concentrated-liquidity AMM forked from Uniswap V3. It adds per-tick tracking of staked liquidity together with a gauge that holds the staked positions and pays them emissions instead of swap fees, a pluggable hook that sets the swap fee, and a permissionless staker for third-party incentive campaigns.

The most critical subjects covered in our audit are escalation of privileges, fairness of the reward distribution, and liveness. Security regarding escalation of privileges is good, with a smaller issue in [Emergency Council with Cap Admin Rights Can Only Zero Caps](#). The fairness of the distribution is good, with the caveat in [Liquidity in Range Past an Incentive's End Still Earns Rewards](#). The liveness is not sufficient at the reviewed commit as user funds can get stuck on chain suspension, see [Chain suspension locks staked NFTs with pending rewards](#).

The general subjects covered are integration testing, documentation & specification, and error handling. Security regarding integration testing is low in the current version. The contracts are not integrated into the shared integration testing suite, so their functionality is not guaranteed. [setEmissionCap reverts: LeafVoter has no emergencyCouncil](#) and [Interface Mismatch of IGauge](#). Security regarding documentation & specification is high, with extensive documents available, though the code deviates from them in places, see [Claim Amount Is Remapped in Undocumented Ways](#). Security regarding error handling is improvable. Velodrome has chosen to wrap a number of external calls in `try / catch` to guarantee liveness to prevent, for example, a math formula in one of the fee modules reverting and then blocking trades. That design choice gives the attacker the optionality to enter the failure branch if they can just trigger a revert in the surrounding code. As the `catch` cannot distinguish revert reasons, this is hard to prevent, since an attacker can always cause a failure by choosing a transaction gas limit that starves the callee and selects the branch. See [Hardcoded gas stipends cannot adapt to gas repricing](#) and [Insufficient gas grieving to trigger the else branch](#). `Version 2` removed the gas caps at these sites, though the calls stay wrapped in `try / catch`, so a caller who controls the transaction gas can still starve the callee under the 63/64 rule.

In contrast to a regular audit process, the codebase was not fixed at a single commit but evolved continuously during the review and as such the latest version of the code was not covered extensively. Together with the improvable testing suite, we believe an above-average number of undiscovered issues remain.

In summary, we find that the codebase provides an improvable level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the MetaDEX Slipstream repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	17 June 2026	17330331b911fca44e5c8535b601778d7f7f3b48	Initial Version
2	10 August 2026	4d013bad41be21f1933c73a5c7ab9856baa624aa	Extended Scope

Version 1 and **Version 2** versions were covered by a single continuous review with no fix round in between. A finding tagged **Version 2** may therefore have been present in **Version 1** as well. The tag names the version the finding is written against and is chosen for readability. It does not indicate when or how the issue was found. Further, some patches for findings are already partially applied, but are not in the current report due to time constraints.

For the Solidity smart contracts, the compiler version `0.7.6` was chosen.

The following files were reviewed in full:

```
contracts/
├── core/
│   ├── indexation/
│   │   └── CLFactoryIndexation.sol
│   ├── libraries/
│   │   └── hook/
│   │       └── SwapHookLib.sol
│   ├── gauge/
│   │   ├── CLGauge.sol
│   │   ├── CLGaugeFactory.sol
│   │   └── Gauge.sol
│   └── staker/
│       ├── SlipstreamStaker.sol
│       └── libraries/
│           ├── IncentiveId.sol
│           ├── NFTPositionInfo.sol
│           ├── RewardMath.sol
│           └── TransferHelperExtended.sol
```

The files below were reviewed as a *diff* audit, meaning only the changes relative to the reference commit `479f512de8c72b863168d5b46788d21618d4e59f` were evaluated:

```
contracts/
├── core/
│   ├── CLFactory.sol
│   └── CLPool.sol
└── periphery/
```

 `└─ NonfungiblePositionManager.sol`

2.1.1 Excluded from scope

All other contracts not explicitly mentioned are outside the scope of this review.

For the diff-audited files, we assume that the Slipstream codebase at the reference commit is correct and secure.

Further, the economic design and the correct choice of parameters are out of scope of this review.

The contracts are expected to be deployed on the Optimism and Base L2s. Other chains are not covered by this review, and the differences in their behavior must be carefully considered before deploying there.

3 System Overview

Slipstream is Velodrome's concentrated-liquidity AMM. The core pool and swap logic is a fork of Uniswap V3 (`v3-core` / `v3-periphery`), renamed from `UniswapV3*` to `CL*` but keeping the original callback names (`uniswapV3SwapCallback` , `uniswapV3MintCallback` , `uniswapV3FlashCallback`) for integrator compatibility. On top of this, the repository adds a gauge system that lets staked positions earn Velodrome emissions instead of swap fees, and a standalone incentive staker that lets any third party run reward campaigns on any pool.

The codebase splits into three main parts:

- **Pools:** liquidity, swaps, fee accounting, NFT positions. Upstream Uniswap V3 code plus Velodrome-specific hooks.
- **Gauges:** emissions distribution to staked positions and routing of their fees to voters. Velodrome-native.
- **Staker:** a separate, permissionless incentive-campaign contract modeled on Uniswap's `v3-staker`.

3.1 Pools

`CLFactory` deploys pools as EIP-1167 clones of a single `CLPool` implementation, deterministically salted on (`token0` , `token1` , `tickSpacing`). Only tick spacings explicitly enabled by the factory owner can be used. The shipped set is `1 / 50 / 100 / 200 / 2000` with default fees of 1/5/5/30/100 bps respectively (fees are expressed in pips, i.e. millionths, matching Uniswap V3 convention rather than Velodrome's usual bps). Pools are created independently of gauges and always come initialized with a starting price. After deployment, each pool is registered in the `factoryRegistry` via `registerTarget` , so a gauge can be created for it immediately.

The factory accepts a `voter` parameter in its constructor. If the factory is deployed with `voter == address(0)` , the `setVoter()` setter allows it to be set exactly once after deployment. This setter is callable only by the factory owner, and only while `voter` remains the zero address. The `factoryRegistry` is supplied directly as a constructor argument.

`CLPool` extends the standard Uniswap V3 tick/liquidity/oracle model with a second, per-tick liquidity value: **staked liquidity**. When a position is staked into its gauge, the pool moves the position's liquidity from being owned by the `NonfungiblePositionManager` to being owned virtually by the gauge, while keeping it in the same tick range.

This lets the pool track, per tick, how much of the active liquidity belongs to staked (gauge) positions versus unstaked (fee-earning) positions, and split swap/flash fees between the two accordingly. A configurable **unstaked fee** is levied on the portion of fees earned by unstaked LPers. The rest of staked-position fees are collected in bulk by the gauge.

`CLPool` exposes a `POOL_TYPE` constant (`bytes32("CL")`) used by the meta-router for dispatch. The pool also provides three fee views: `beforeSwapFee(SwapParams)` and `afterSwapFee(SwapParams, AfterSwapParams)` return the fee applied at each stage of the swap loop, and `flashFee(FlashParams)` returns the flash loan fee. A `fee()` view is retained for backward compatibility and delegates to `beforeSwapFee` with `msg.sender` as the caller, so discount lookups continue to work through it.

3.2 Fee modules

`CLPool` does not hard-code its swap fee. The swap fee is determined by a pluggable swap hook set on the factory. Each pool has an associated hook address, defaulting to the factory-wide hook unless overridden per pool. The hook implements the `ISwapHook` interface and is invoked by `SwapHookLib` at the start and end of each swap and at the start of each flash loan.

If the hook returns a fee above `_SWAP_FEE_CEIL` (10%), the fee is clamped to the ceiling. If the hook is unset or reverts, the fee falls back to the pool's tick-spacing default. `SwapHookLib` also exposes three read-only helpers, `getBeforeSwapFee`, `getAfterSwapFee`, and `getFlashFee`, that back the corresponding fee views on `CLPool`.

The unstaked fee is controlled separately via a pluggable `unstakedFeeModule` set by the `unstakedFeeManager`. If no module is configured, the factory applies a default unstaked fee of 10%.

All fee roles (`swapFeeManager`, `unstakedFeeManager`) are independent, factory-owner-appointed addresses, distinct from the factory `owner` itself. The `swapFeeManager` controls per-pool hook assignments on `CLFactory` via `setDefaultSwapHook()` and `setPoolSwapHook()`.

3.3 Pool discovery

`CLFactoryIndexation` maintains append-only on-chain indexes covering all pools with creation timestamps, all tokens ever used, and pools grouped by token. These are populated automatically whenever `createPool` succeeds. They exist purely for discoverability (e.g. by off-chain indexers or other contracts) and are paginated to avoid unbounded reads.

3.4 Periphery

`NonfungiblePositionManager` is the standard entry point for LPs: it wraps pool positions as ERC-721 tokens and keeps separate fee accounting for staked and unstaked positions so that a staked position's owed fees stay at zero while staked.

3.5 Gauges

A gauge is created per pool by `CLGaugeFactory`, callable only by the external `gaugeManager` address. `createGauge()` deploys the gauge as an EIP-1167 clone and creates the associated `VotingRewardsManager` via `VotingRewardsFactory`. All state writes complete before any external call, following checks-effects-interactions order. The CREATE2 salt is keyed on the pool address alone. `CLGauge.initialize()` requires `msg.sender == gaugeFactory` to prevent re-initialization by arbitrary callers.

Depositing an NFT into the gauge (`deposit`) settles any pending fees to the depositor, moves the position's liquidity to staked accounting in the pool, and starts emissions accrual. Withdrawing (`withdraw`) reverses this and always auto-claims pending rewards. There is no way to claim after withdrawal, so rewards must be collected while staked. Withdrawal is always possible, even from a gauge that has been capped to zero emissions.

Reward accrual uses a pull model driven by the `LeafVoter`. The voter maintains a monotonically increasing cumulative reward share for each gauge, accessible via `LeafVoter.settleGauge(gauge)`. This is a public, permissionless function that settles the voter's internal accounting and returns the running total. It returns zero for unregistered addresses without reverting.

`CLGauge` stores a `lastCumulativeRewardShare` cursor. Whenever the pool calls `_settleToBlock()`, it calls `gauge.settleGauge()`. The gauge calls `voter.settleGauge(address(this))`, takes the delta since its cursor, advances the cursor, and returns the amount as `_rewardsToDistribute`. The pool feeds this directly into the global reward-growth accumulator. A position's claimable emissions depend on how much of that growth occurred while its tick range was active.

When `setGaugeAndPositionManager()` first connects a gauge to a pool, the pool calls `gauge.settleGauge()` immediately. Any emissions the voter allocated before the connection are pulled at this point. Since no liquidity is staked yet, they accrue into rollover.

If the active tick has no staked liquidity, emissions accumulate as rollover in the pool. `CLGauge` exposes a public `forfeitRollover()` function. It calls `pool.settleToBlock()`, which returns and clears the rollover, then forwards it to `voter.forfeitEmissions()`. This lets anyone return idle emissions to the voter's surplus pool.

The `earned()` view estimates claimable rewards by calling `voter.projectedCumulativeRewardShare(address(this))`, a read-only projection at the current block, and diffing it against `lastCumulativeRewardShare`.

Two mechanisms sit on top of the base reward calculation, both configured on `CLGaugeFactory`:

- **Early-unstake penalty:** if a position's rewards are claimed before it has been staked for a minimum number of blocks, a configurable fraction of the reward is forfeited back to the Voter instead of being paid out. This discourages stake-claim-unstake cycling around reward distribution.
- **Referral fee:** a gauge can have a referral address and share (in pips) configured, which redirects part of every claim to the referrer.

Each gauge also has a weekly **emission cap**, defaulting to a factory-wide value unless overridden. A cap of zero stops both new emissions and fee collection for that gauge. `CLGaugeFactory` gates these controls with OpenZeppelin `AccessControl` roles rather than a single owner: a cap admin (plus a narrower-range cap operator) controls emission caps, a referral admin controls referral configuration, and a penalty admin controls the penalty rate and per-gauge minimum stake duration.

3.6 Slipstream Staker

`SlipstreamStaker` is a separate, permissionless contract, independent of the gauge system, modeled on Uniswap's `v3-staker`. It lets anyone fund a time-boxed incentive (reward token, pool, start/end time, refund address) on any Slipstream pool and lets LPs deposit their position NFTs and opt into one or more incentives to earn those rewards, proportional to the time their position spent in range. Positions are deposited into the staker itself (separately from gauge staking) and can be staked into multiple incentives at once.

To avoid incentive campaigns fighting Velodrome's own emissions, the staker's owner can restrict incentive creation to periods when the pool's gauge is not actively emitting (its emission cap is zero or its gauge inactive), or allow creation at any time.

Like the gauge, it applies an early-unstake penalty: unstaking before a minimum number of blocks forfeits a configurable share of the earned reward, which is added back to the incentive's pool rather than paid to the staker or the unstaker.

Once an incentive's end time plus a fixed refund delay has passed and no positions remain staked to it, its creator (or designated refundee) can reclaim whatever reward was never distributed, including anything forfeited to penalties. Reward accounting is keyed to the amount requested rather than the amount received, so tokens that do not transfer exactly, such as fee-on-transfer, deflationary or rebasing tokens, are not supported as reward tokens.

4 Trust Model

No contract in scope uses a proxy pattern. `CLPool` and `CLGauge` are EIP-1167 minimal clones of immutable implementation contracts. `CLGaugeFactory`, `NonfungiblePositionManager`, and `SlipstreamStaker` are all non-upgradeable.

4.1 Roles

CLGaugeFactory - CAP_ADMIN_ROLE

- Trust level: Partially trusted.
- Permissions: Sets the global default emission cap and the per-gauge emission cap for any registered gauge to any value, including zero. Also sets the `[operatorMinCap, operatorMaxCap]` range that bounds `CAP_OPERATOR_ROLE`.
- Worst-case impact: Setting any gauge's cap to zero stops new emission accrual and prevents the gauge from collecting fees via `collectFees`. However, it does not disable the unstaked fee. Unstaked LPs continue to pay the unstaked fee into the pool's `gaugeFees` slot, but the gauge refuses to collect it while its cap is zero. These fees remain locked until the cap is raised. Setting the default cap to zero has the same effect on all gauges still carrying the default-cap sentinel. The cap admin cannot steal LP principal or redirect already-accrued rewards.

CLGaugeFactory - CAP_OPERATOR_ROLE

- Trust level: Partially trusted.
- Permissions: Can update per-gauge emission caps. The range check `[operatorMinCap, operatorMaxCap]` is enforced only when the new cap is not the `DEFAULT_CAP_INDICATOR` sentinel (`type(uint128).max`). A `CAP_OPERATOR_ROLE` holder can therefore restore a gauge's cap to `DEFAULT_CAP_INDICATOR`, which bypasses the range check and causes the gauge to inherit the factory-wide default cap. Cannot set a cap to zero, since `operatorMinCap` must be nonzero.
- Worst-case impact: Can raise or lower emission throughput for individual gauges within the operator range, distorting rewards across pools. By restoring `DEFAULT_CAP_INDICATOR`, an operator can effectively uncap any gauge even when `operatorMaxCap` is set to a restrictive value, as long as the factory default cap is higher. Cannot disable a gauge or redirect LP funds.

CLGaugeFactory - REFERRAL_ADMIN_ROLE

- Trust level: Partially trusted.
- Permissions: Sets the per-gauge referral address and referral share (in pips, capped at `maxShareCap`, default 5%). Also sets `maxShareCap` itself (capped at 100%).
- Worst-case impact: Can redirect up to `maxShareCap` of every claim's gross emissions from any gauge to an arbitrary address. If `maxShareCap` is first raised to 100% and then the referral share for a gauge is set to 100%, all future emission claims from that gauge are diverted entirely to the referral address. Cannot affect LP principal or rewards already accrued inside the pool.

CLGaugeFactory - PENALTY_ADMIN_ROLE

- Trust level: Partially trusted.
- Permissions: Sets the global early-exit penalty rate (in pips, up to 100%) and the global minimum stake duration (in blocks, bounded by the constructor-immutable `maxMinStakeBlocks`). Can also set a per-gauge minimum stake duration override.
- Worst-case impact: Setting the penalty rate to 100% and the minimum stake window to `maxMinStakeBlocks` causes any LP that claims before that window expires to forfeit their entire accrued

reward back to the Voter. Cannot affect LP principal held in the NFT, which is always withdrawable regardless of penalty configuration.

NonfungiblePositionManager - Owner

- Trust level: Partially trusted.
- Permissions: Can replace `tokenDescriptor` (the on-chain NFT metadata/artwork renderer) with any address. Can transfer ownership.
- Worst-case impact: A malicious `tokenDescriptor` causes `tokenURI` calls to revert or return arbitrary data, degrading off-chain display and integrators that rely on on-chain metadata. Does not affect LP principal, fee accrual, or token transfers. The owner has no power over pool accounting or position liquidity.

CLFactory - swapFeeManager (factory-appointed)

- Trust level: Partially trusted.
- Permissions: Controls the default and per-pool swap hook assignments on `CLFactory` via `setDefaultSwapHook` and `setPoolSwapHook`.
- Worst-case impact: Can route all swaps through an arbitrary hook for any pool. A hook setting fees at the ceiling (10%) extracts the most from every swap. A hook setting fees to zero removes LP fee revenue. Hooks with an `afterSwap` callback write fees unconditionally to the pool's `gaugeFees` slot without checking whether a gauge exists. Assigning a fee-collecting hook to a pool with no gauge permanently locks those fees, since only the gauge can call `collectFees`. The swapFeeManager is trusted not to configure fee-collecting hooks on pools without gauges.

SlipstreamStaker - Owner

- Trust level: Partially trusted.
- Permissions: Sets the incentive `creationMode` (`ALWAYS` or `WHEN_NOT_EMITTING`), the global penalty rate and minimum stake duration (bounded by the constructor-immutable `maxMinStakeBlocks`), and the `microsPerBlock` conversion factor used for penalty timing. Can transfer ownership.
- Worst-case impact: Setting a 100% penalty rate with the maximum stake window causes LPs that exit early to forfeit their entire reward back into the incentive pot, which is recoverable by the refundee and not the staker owner. Setting `creationMode` to `ALWAYS` permits incentive campaigns to run alongside active gauge emissions. The owner cannot access deposited NFTs, drain rewards credited to LP accounts, or modify existing incentives. The staker has no on-chain validation of reward token transfer behavior. Reward accounting credits the amount requested by the incentive creator, not the amount actually received. A fee-on-transfer reward token therefore lets a new `createIncentive` call draw tokens from the shared balance that other incentives rely on. The owner is trusted to enforce token compatibility off-chain or to add on-chain restrictions if this risk is unacceptable.

Liquidity Providers / End Users

- Trust level: Untrusted.
- Permissions: Can mint, increase, and decrease positions through `NonfungiblePositionManager`. Can deposit and withdraw NFTs from `CLGauge` and `SlipstreamStaker`. Can create incentive campaigns in the staker (subject to creation-mode restrictions), stake and exit those incentives, and claim earned rewards.
- Worst-case impact: An LP can only affect their own positions and claimed rewards. They cannot influence other users' accounting, pool state, or protocol configuration. One exception involves `NonfungiblePositionManager.refundETH()`, which is called at the end of `mint`, `increaseLiquidity`, and the swap entry points. It forwards the contract's entire ETH balance to `msg.sender`. If any ETH is left in the contract from a prior payable call (e.g., an incomplete multicall), it is forwarded to the next caller of any of these functions. A contract integrator without a payable fallback will revert on that transfer, bricking those entry points for that integrator until the balance is drained. Any user can trigger this condition by leaving 1 wei in the contract.

4.2 External Dependencies

Voter / LeafVoter The gauge system is fully subordinate to the external Voter for its emissions lifecycle. The Voter drives gauge creation (via `gaugeManager`), pushes reward rates via `settleGauge`, mints emissions to LPs and referrers via `mintEmissions`, and accepts forfeited rollover via `forfeitEmissions`. It also exposes the `emergencyCouncil` address that `CLGaugeFactory` queries directly to authorize emergency cap-zero operations. The cumulative reward share the Voter reports on `settleGauge` directly determines how much of the reward token is minted to LPs. A Voter that over-reports mints excess tokens, while one that under-reports or stops reporting silently starves gauges of emissions. Unavailability of the Voter does not block deposits or withdrawals but prevents new emission accrual and fee collection, because `forfeitEmissions` and `mintEmissions` are called during every claim. A compromised Voter can halt all gauge emissions, drain the protocol's emission budget by inflating the cumulative reward share delta, or use the `emergencyCouncil` it exposes to simultaneously zero every gauge's emission cap, freezing fee collection for all staked LPs.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 CLGauge Accepts Donations from Anyone

System Consideration	Version 1
Contracts that claim emissions from <code>CLGauge</code> on behalf of a user should consider that an account does not control the set of NFTs staked for it. <code>Gauge.depositFor()</code> credits a stake to an arbitrary <code>owner</code>, so any party can grow a chosen account's stake set with dust positions. An integration must expect that someone else deposited on that account's behalf, and should keep its own accounting of the positions it staked rather than trusting the set of positions stored in the gauge on behalf of them. As a side effect of that mechanism, the unbounded entry points can be pushed past the block gas limit. <code>claimEmissions(address, address)</code>, <code>earned(address)</code> and <code>stakedValues()</code> read the whole enumerable set and operate on every NFT in it, so enough donated positions make them run out of gas. An integrating contract must therefore call <code>claimEmissions(address, address, uint256[])</code> or <code>earned(address, uint256[])</code> and evaluate only on ids it knows of.	

SC2 Security Considerations for Pricing a Swap Fee on Trade Size

System Consideration	Version 1
<code>beforeSwap()</code> receives <code>amountSpecified</code> before the swap loop runs. This is the requested trade amount, and can be more than the amount filled when the pool runs out of liquidity or, more importantly, when the limit price <code>sqrtPriceLimitX96</code> is reached. The <code>beforeSwap()</code> hook should not charge a fee based on the specified value. If large swaps are priced cheaper than small ones: A trader can request a large amount to receive a low fee and combine it with a tight limit price, so that only the intended amount actually trades while paying an artificially low fee. Similarly, if large swaps are more expensive than small ones: A trader requesting a large trade and accepting a high fee in return could get sandwiched so that the limit price is reached earlier than expected, and end up paying a large trade's fee on a small fill. A hook whose fee depends on trade size should therefore be implemented in <code>afterSwap()</code> rather than in <code>beforeSwap()</code>. The filled amount is available there, as <code>amountSpecified</code> less <code>amountSpecifiedRemaining</code>.	

SC3 Security Considerations for the Not-Emitting Restriction

System Consideration	Version 1
<code>CreationMode.WHEN_NOT_EMITTING</code> stops an incentive from being created while the pool's gauge is emitting, but <code>_gaugeEmitting()</code> returns false whenever the voter reports no gauge, which includes a pool the factory has not deployed yet. A campaign created that early passes the gate and keeps running once the gauge begins emitting.	

Pool addresses are deterministic, so campaigns can be prepared for pools that do not exist yet. Only `maxIncentiveStartLeadTime` bounds how far ahead of its start a campaign may be created, so it must be chosen small enough to make that impractical.

SC4 Security Considerations When Claiming Gauge Emissions

System Consideration	Version 1
----------------------	-----------

`CLGauge` charges an early-unstake penalty on the emissions of any position claimed or withdrawn while `block.number` is below its `depositBlock` plus `minStakeBlocks`. At the factory defaults of `DEFAULT_MIN_STAKE_BLOCKS = 5` and `DEFAULT_PENALTY_RATE = MAX_PIPS`, the whole balance gets slashed.

Nothing in the call reports it. `claimEmissions()` and `withdraw()` return no amount received. An integrating contract must therefore compare each position's `depositBlock` against `CLGaugeFactory.minStakeBlocks()` before claiming.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

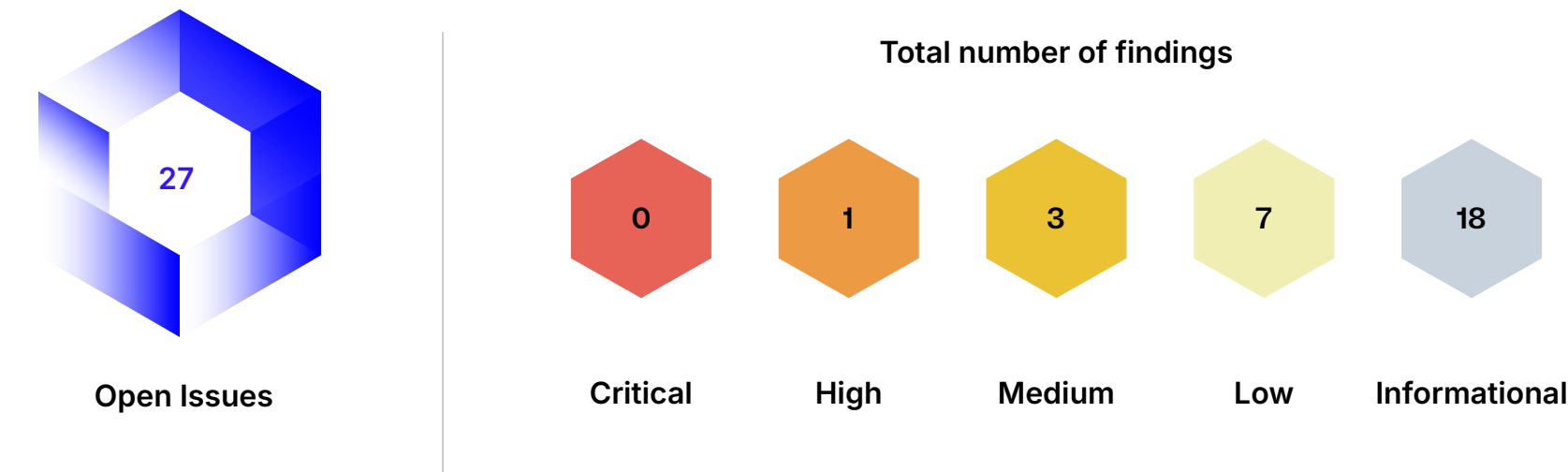
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the [Terminology](#) section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

High	#001	setEmissionCap Reverts: LeafVoter Has No emergencyCouncil	Open	○
Medium	#002	Chain Suspension Locks Staked NFTs with Pending Rewards	Open	○
Medium	#003	Hardcoded Gas Stipends Cannot Adapt to Gas Repricing	Code Corrected	✓
Medium	#004	Insufficient Gas Griefing to Trigger the Else Branch	Open	○
Low	#005	A Hook Cannot Set a Zero Swap Fee	Acknowledged	⊖
Low	#006	A Zero Refundee Makes Residual Rewards Unrecoverable	Open	○
Low	#007	Emergency Council with Cap Admin Rights Can Only Zero Caps	Open	○
Low	#008	Ending an Incentive Campaign Can Be Griefed	Open	○
Low	#009	Griefing via Initial Price	Acknowledged	⊖
Low	#010	Liquidity in Range Past an Incentive's End Still Earns Rewards	Open	○
Low	#011	setDefaultCap Retroactively Reprices Unsettled Gauge Accrual	Open	○
Info	#012	A Sequencer Outage Can Lead to Slashing	Open	○
Info	#013	Any Contract Is Accepted as the Reward Token of an Incentive	Open	○
Info	#014	Claim Amount Is Remapped in Undocumented Ways	Open	○
Info	#015	CLPoolTape Called by Swap Hook Instead of Pool Directly	Open	○
Info	#016	Draining a Tick Range Temporarily Locks a Zero-Liquidity NFT	Open	○
Info	#017	Gauge Clone Is Left Uninitialized Across an External Call	Open	○
Info	#018	Incentive Top-Ups Are Emitted as Incentive Creations	Open	○
Info	#019	Inconsistent NatSpec on SwapParams	Code Corrected	✓
Info	#020	Interface Mismatch of IGauge	Open	○
Info	#021	Magic Value of 0 for Reward Growth	Open	○

Info	#022	Module Unstaked Fees Can Reach 100%, Twice the Factory Cap	Open	☐
Info	#023	Package.json Files Entry Omits Contracts/periphery/libraries	Open	☐
Info	#024	Read-Only Re-Entrancy in NonfungiblePositionManager's Collect	Open	☐
Info	#025	Redundant Account Argument in withdrawTokenFrom	Open	☐
Info	#026	Referral Parameters Are Decoded Without a Length Check	Open	☐
Info	#027	Reward Is Not Bounded by the Incentive's Unclaimed Balance	Open	☐
Info	#028	SlipstreamStaker Constructor Misses Zero Address Checks	Open	☐
Info	#029	withdrawToken Accepts the Zero Address as Recipient	Open	☐



7.2 Descriptions

#001 setEmissionCap Reverts: LeafVoter Has No emergencyCouncil

Correctness	High	Version 1	Open 
-------------	------	-----------	--

`CLGaugeFactory.setEmissionCap` reads the emergency-council address as its first action:

```
address emergencyCouncil = ILeafVoter(leafVoter).emergencyCouncil();
if (msg.sender == emergencyCouncil) { ... return; }
if (hasRole(CAP_ADMIN_ROLE, msg.sender)) { ... return; }
if (hasRole(CAP_OPERATOR_ROLE, msg.sender)) { ... return; }
```

The project's `ILeafVoter` interface declares `emergencyCouncil()` external view returns (address), but the real `LeafVoter` does not implement this function. It manages its emergency council via `EMERGENCY_COUNCIL_ROLE` on `AccessControlEnumerable`, and exposes no `emergencyCouncil()` view. With no matching selector and no fallback on the `LeafVoter`, the external call reverts, and `setEmissionCap` becomes unreachable for all callers — emergency council, `CAP_ADMIN_ROLE`, and `CAP_OPERATOR_ROLE` alike. No gauge cap can be set, changed, or zeroed after deployment.

#002 Chain Suspension Locks Staked NFTs with Pending Rewards

Security	Medium	Version 1	Open 
----------	--------	-----------	--

`CLGauge._withdraw()` unconditionally calls `_claimPositions()` as part of every withdrawal:

```
_claimPositions(account, _tokenIds, recipient);
pool.stake(-(uint256(liquidityToStake).toInt256()).toInt128(), tickLower, tickUpper);
...
nft.safeTransferFrom(address(this), msg.sender, tokenId);
```

`_claimPositions` settles the pool and, when a position has accrued LP rewards, calls `voter.mintEmissions()`:

```
if (_amounts.length > 0) {
    voter.mintEmissions(_recipients, _amounts);
}
```

`LeafVoter.mintEmissions` carries a `chainIsActive` modifier that reverts with `ChainNotActive` whenever `chainStatus != Active`. When the chain is suspended — which zeroes `emissionsPerVP` and stops new accrual — any withdrawal attempt on a position with unclaimed LP rewards in `rewardGrowthGlobalX128` fails at this call, blocking the NFT transfer.

`forfeitEmissions` (called for rollover and penalties) carries no such modifier and is unaffected. Only positions with a non-zero LP reward component are locked; positions that accrued nothing, or whose rewards were fully claimed before suspension, can still be withdrawn.

There is no withdrawal path that bypasses `_claimPositions`, and `claimEmissions` routes through the same function, so the accrued rewards cannot be drained first. The lock persists for the duration of the suspension. Because the `Suspended → Active` transition is the only exit from `Suspended`, the chain can always be resumed; however, governance may delay that resumption arbitrarily, extending the lock.

#003 Hardcoded Gas Stipends Cannot Adapt to Gas Repricing

Security

Medium

Version 1

Code Corrected ✓

Every gas-limited external call in scope sizes the callee's budget with a compile-time constant. None of these values has a setter, and the constants section holding the hook values carries a `// TODO(future PRs/commits): review and assess sanity of these values` note. The affected sites are:

1. `SwapHookLib._safeCall()` (`_SAFE_CALL_GAS_LIMIT`, `500_000`): a `private` constant forwarded to the swap hook on all three hook entry points, `beforeSwap()`, `afterSwap()` and `beforeFlash()`.
2. `CLFactory.getSwapFee()` (`200_000`): a literal forwarded to `swapFeeModule`.
3. `CLFactory.getUnstakedFee()` (`200_000`): a literal forwarded to `unstakedFeeModule`.
4. `CLFactory.createPool()` (`gasleft()`): forwards all remaining gas, but fixes the work the callee has to perform through `DEFAULT_POOL_TAPE_CARDINALITY`, an `internal` constant set to `100`.

```
(bool _success, bytes memory _data) =
    _swapHook.excessivelySafeCall(_SAFE_CALL_GAS_LIMIT, _SAFE_CALL_RETURN_DATA_SIZE,
        _hookData);
```

In each of these instances an out-of-gas failure in the callee is swallowed silently rather than reverting, so a stipend that becomes too small degrades behavior without any on-chain signal. `SwapHookLib` is inlined into the pool implementation, so its limit is fixed for every pool cloned from that implementation and can only be changed by deploying a new implementation. Existing pools keep the old value.

The Glamsterdam upgrade raises the cost of zero-to-nonzero storage writes, so stipends sized against today's costs can fall short afterward. The cost of a zero-to-nonzero storage write is expected to increase from 20,000 to 100,000 gas, which turns a `200_000` stipend from ten such writes into two.

Note that Glamsterdam is also expected to add a separate `state_gas_reservoir` for storage writes, populated by setting a gas limit above the per-transaction cap of roughly 16 million gas, so the affected calls would not be permanently bricked. They would however require the caller to set a very large initial gas limit to succeed, with potentially problematic UX consequences, and insufficient gas griefing attacks would become trivially achievable.

The pre-warm in `createPool()` is exposed the same way from the other side. Its cardinality target of `100` fixes the number of observation slots the pool tape must write in one call, and because the value is a constant it cannot be lowered to fit a repriced cost. The pool tape implementation is not part of this scope, so the actual cost of that call cannot be bounded here.

More generally, because the stipends are hardcoded, the protocol cannot raise them in response to any later repricing of state costs, which is always possible and could lead to more critical flaws.

Code Corrected

The hardcoded stipends were removed. The hook entry points in `SwapHookLib` and the fee module lookup in `CLFactory` now use `try / catch` passing 63/64 of the available gas.

#004 Insufficient Gas Griefing to Trigger the Else Branch

Security

Medium

Version 1

Open 

In `SwapHookLib` and `CLFactory`, several external calls are wrapped in `excessivelySafeCall` or `excessivelySafeStaticCall` and the result is handled by branching on success. The gas argument is only an upper bound on what reaches the callee. Per EIP-150 the callee can use at most `min(requested, gasleft() - gasleft() / 64)`, and the EVM does not revert when less than the requested amount is available. These wrappers also cannot tell a genuine revert from an out-of-gas, as the EVM does not expose the difference.

```
(bool success, bytes memory data) =
    swapFeeModule.excessivelySafeStaticCall(200_000, 32,
        abi.encodeWithSelector(IFeeModule.getFee.selector, pool));
if (success) { /* use module fee */ } // else fall back to the default fee
```

The gas remaining at the call site depends on the top-level transaction gas limit, which the sender of the transaction chooses. A malicious caller can pick a limit such that `gasleft()` at the call site is below the requested stipend. The callee then runs out of gas, while the one sixty-fourth retained by the calling frame is still enough to finish the outer logic. The caller is left with a free choice between the callee's result and the fallback, and takes whichever is more favorable. There are multiple sites where this is possible:

1. `SwapHookLib.beforeSwap() ( 500_000 )`: falls back from the hook's fee to `tickSpacingToFee[tickSpacing]`, which then prices the entire swap.
2. `SwapHookLib.beforeFlash() ( 500_000 )`: falls back from the hook's flash fee to `tickSpacingToFee[tickSpacing]`.
3. `SwapHookLib.afterSwap() ( 500_000 )`: drops the post-swap fee that would have been credited to `gaugeFees`.
4. `CLFactory.getUnstakedFee() ( 200_000 )`: falls back to `defaultUnstakedFee`. `CLPool` reads this through `unstakedFee()` on every fee-accruing step of a swap or flash, so the fallback shifts the fee split between staked and unstaked liquidity.
5. `CLFactory.getSwapFee() ( 200_000 )`: falls back to `tickSpacingToFee[tickSpacing]` and is exposed to integrators through `CLPool.fee()`. Within the module, `DynamicSwapFeeModule._getDynamicFee()` additionally catches a failing `observe()` and returns a zero surcharge, so starving that inner call lowers the reported fee without failing the outer read.
6. `CLFactory.createPool() ( gasleft() )`: an out-of-gas in the pool tape's `increaseObservationCardinalityNext()` silently drops the initial observation-cardinality bump.

In practice, whether this is exploitable depends heavily on the callee's actual gas use and on the gas cost of the execution thereafter, which still has to succeed. Both are notoriously hard to calculate, though. Where it is feasible the caller chooses the fee that prices their own swap or flash loan, the split between staked and unstaked liquidity, and whether the gauge is credited a post-swap fee at all, with nothing on-chain flagging that the callee never ran.

Changes in Version 2

The gas caps were removed. The calls into the swap hook and into the unstaked fee module now use `try / catch` without a gas limit, passing 63/64 of the available gas.

#005 A Hook Cannot Set a Zero Swap Fee

Design	Low	Version 1	Acknowledged
--------	-----	-----------	--------------

In `SwapHookLib`, `beforeSwap()`, `afterSwap()` and `beforeFlash()` all reach the hook through `_safeCall()`. That helper returns two values, a flag saying whether a usable fee was obtained and the fee itself.

```
(bool _success, bytes memory _data) =
    _swapHook.excessivelySafeCall(_SAFE_CALL_GAS_LIMIT, _SAFE_CALL_RETURN_DATA_SIZE,
        _hookData);

if (_success && _data.length == _SAFE_CALL_RETURN_DATA_SIZE) {
    uint256 _feeRaw = abi.decode(_data, (uint256));
    if (_feeRaw <= _SWAP_FEE_CEIL) {
        _fee = uint24(_feeRaw);
        _okFee = _success;
    }
}
```

The flag is the only thing that separates a failed call from a successful one. There are three scenarios:

- 1. Hook returns a valid non-zero fee: flag true, fee above zero.
- 2. Hook fails, returns malformed data, or returns a fee above `_SWAP_FEE_CEIL`: flag false, fee zero.
- 3. Hook returns a valid fee of zero: flag true, fee zero.

In the third scenario, only `beforeFlash()` would use the flag. `beforeSwap()` checks that the fee is above zero and would then treat the hook as having failed, so it replaces the hook's zero fee with the factory tick-spacing fee. `afterSwap()` likewise ignores the flag and acts only when the fee is above zero, though there skipping the branch and applying a zero fee have the same effect.

Having each hook resolve the same helper's result by its own rule makes future changes to the hook interface more complicated as one has to reason about every entry point separately.

Acknowledged

Velodrome has provided the following reasoning to keep the code unchanged

Regarding inability to return 0 fee on swap methods, this is expected and acceptable.

#006 A Zero Refundee Makes Residual Rewards Unrecoverable

Design	Low	Version 1	Open
--------	-----	-----------	------

`createIncentive` validates the reward amount, timing, and duration but does not check that `key.refundee` is a non-zero address. `endIncentive` sends the unclaimed balance directly to `key.refundee`:

```
TransferHelperExtended.safeTransfer(address(key.rewardToken), key.refundee, refund);
```

Because the incentive ID is the hash of the full key, an incentive created with `refundee == address(0)` cannot be superseded by one with a valid refundee. After `endTime + REFUND_DELAY`, any tokens remaining in `totalRewardUnclaimed + totalRewardSlashed` are sent to the zero address and permanently lost.

#007 Emergency Council with Cap Admin Rights Can Only Zero Caps

Design

Low

Version 1

Open

`setEmissionCap` checks the emergency-council identity before the `CAP_ADMIN_ROLE` grant and returns early from the emergency branch:

```
address emergencyCouncil = ILeafVoter(leafVoter).emergencyCouncil();
if (msg.sender == emergencyCouncil) {
    require(_cap == 0, 'NA');
    ...
    return; // CAP_ADMIN branch is never reached
}
if (hasRole(CAP_ADMIN_ROLE, msg.sender)) { ... }
```

An address that simultaneously holds `emergencyCouncil` status and `CAP_ADMIN_ROLE` is always routed to the emergency branch, which requires `_cap == 0`. It cannot use its `CAP_ADMIN_ROLE` powers to set non-zero caps. The condition is recoverable by granting `CAP_ADMIN_ROLE` to a separate address, but the ordering makes the dual-role configuration silently restrictive.

#008 Ending an Incentive Campaign Can Be Griefed

Design

Low

Version 1

Open

`SlipstreamStaker.endIncentive()` refunds the refundee only once the incentive's `numberOfStakes` has been reduced to zero, so a griever can stake a large number of dust positions on an incentive campaign and refuse to unstake. Clearing them without the stakers is possible, since `unstakeToken()` becomes unpermissioned once `endTime + REFUND_DELAY` passes, but it costs the refundee one call per position.

The griever moreover saves gas. `unstakeToken()` performs the pool snapshot, the reward computation and credits `reward - slash` to the position's deposit owner, so whoever clears the incentive pays for that settlement while the staker is left with a credited balance in the `rewards` mapping to collect. Never unstaking therefore imposes a larger cost on the incentive creator than on the griever.

#009 Griefing via Initial Price

Design

Low

Version 1

Acknowledged

`CLFactory.createPool()` is unpermissioned and puts no bound on `sqrtPriceX96` before passing it to `CLPool.initialize()`. The only check is the one inside `TickMath.getTickAtSqrtRatio()`, so a caller can pick any price `sqrt(1.0001^tick)` with the tick anywhere in `[-887,272, 887,271]`. The upper bound is one below `MAX_TICK` because the price can never reach `MAX_SQRT_RATIO`.

```
require(tickSpacingToFee[tickSpacing] != 0);
require(_getPool[token0][token1][tickSpacing] == address(0));
pool =
    Clones.cloneDeterministic({master: poolImplementation, salt: keccak256(abi.encode(token0,
        token1, tickSpacing))});
CLPool(pool)
    .initialize({
        // ...
```

```
_sqrtPriceX96: sqrtPriceX96
});
```

Only one pool can exist for each combination of `token0`, `token1` and `tickSpacing`. Once someone initializes a pool and a skewed price, no replacement can be made anymore, and integrators that compute the pool address are forced to use the pool with the skewed initial price. The enabled tiers are 1, 50, 100, 200 and 2000, so an attacker could create pools for every popular token pair in every tier ahead of time.

The price can only be moved by `swap()`, so fixing the price requires swapping across the whole distance. Each loop iteration in `swap()` covers one full tick-bitmap word or `256 * tickSpacing` ticks if no tick in the word is initialized in `TickBitmap.nextInitializedTickWithinOneWord()`.

Skipping an empty bitmap word is a single cold `SLOAD`, so 2,100 gas. The number of words that need to be iterated through is `wordPos = (tick / tickSpacing) >> 8` over the distance walked.

Take a concrete case. The attacker creates the pool at the minimum price while the honest price sits close to 1 (i.e. near tick 0), which is where a stablecoin pair trades. Fixing the price then means walking from `MIN_TICK` to tick 0, a little over half the tick range.

No position has ever been minted, so every bitmap word on the way is empty. The walk touches `1 + ceil(-MIN_TICK / (256 * tickSpacing))` words, where the ceiling covers the partial word at the `MIN_TICK` end and the `1` is the word holding tick 0:

tickSpacing	storage operations = 887,272 / (256 * s)	rough gas
1	~3,467 cold <code>SLOAD</code>	~13M
50	~71 cold <code>SLOAD</code>	~250k
100	~36 cold <code>SLOAD</code>	~126k
200	~19 cold <code>SLOAD</code>	~67k
2000	~3 cold <code>SLOAD</code>	~11k

Crossing an **initialized tick** is far more expensive. The worst case is a tick never crossed on a pool that has accrued growth. Here crossing could cost at least:

```
6 * 2,100 cold SLOAD, all six slots of Tick.Info
+ 3 * 20,000 SSTORE 0 -> nonzero (feeGrowthOutside0X128,
                                feeGrowthOutside1X128,
                                rewardGrowthOutsideX128)
+ 1 * 2,900 SSTORE nonzero -> nonzero (secondsPerLiquidityOutsideX128)
+ 2 * 100 SSTORE repeat (tickCumulativeOutside, secondsOutside)
= 75,700 gas per crossed tick
```

Glamsterdam makes this worse. EIP-8037 raises the cost of creating new state, taking a fresh storage slot from 20,000 to an estimated 97,920 gas, so the worst-case crossing moves from 75,700 to roughly 310,000.

Integrators must therefore be careful before reading data from or calling into any pool, and check that it was initialized at a sound price or was at least moved to a good state later. Neither the observation history nor the spot price is sufficient on its own, since `OracleLibrary.getOldestObservationSecondsAgo()` can report a long history while `consult()` can average observations that are largely meaningless.

Acknowledged
Velodrome acknowledged this issue and answered

This issue is also present on Uniswap V3. I'm not quite sure if there's any sane mitigation that would allow to keep pool creation permissionless. And there's no external reference price that would allow checking if the passed `sqrtPriceX96` makes actual sense, or if it is inflated.

#010 Liquidity in Range Past an Incentive's End Still Earns Rewards

Correctness

Low

Version 1

Open 

`SlipstreamStaker.unstakeToken()` passes the pool's live `secondsPerLiquidityInsideX128` and `block.timestamp` to `RewardMath.computeRewardAmount()` without clamping either to the incentive's `endTime`:

```
secondsInsideX128 = (secondsPerLiquidityInsideX128 - secondsPerLiquidityInsideInitialX128)
    * liquidity;
uint256 totalSecondsUnclaimedX128 = ((Math.max(endTime, currentTime) - startTime) << 128) -
    totalSecondsClaimedX128;
reward = FullMath.mulDiv(totalRewardUnclaimed, secondsInsideX128,
    totalSecondsUnclaimedX128);
```

`_stakeToken()` accepts new stakes until `endTime`, after which the pool oracle continues accumulating `secondsPerLiquidityInside` for any position still in range. Because the denominator grows with `Math.max(endTime, currentTime)` to match, the per-second rate is unchanged — a position staked just before `endTime` and held in range afterward is paid for time spent after the campaign closed.

The magnitude scales with how long the position is held past `endTime`. A position opened in the last block and left in range through the one-hour `REFUND_DELAY` window earns for that extra hour; on a one-week incentive that is about 0.6% of whatever reward is still unclaimed, though the fraction grows for shorter campaigns.

At the extreme, a staker who enters one second before `endTime` and keeps the position in range for an additional duration equal to the full campaign length `D` accumulates `D + 1` seconds inside while the denominator grows to `2D`, capturing roughly 50% of the unclaimed pot. In both cases the cost is borne by the refundee (reduced residual) and any stakers who remain.

#011 setDefaultCap Retroactively Reprices Unsettled Gauge Accrual

Correctness

Low

Version 1

Open 

`setEmissionCap` checkpoints a gauge before modifying its cap by calling `ILeafVoter(leafVoter).settleGauge(_gauge). setDefaultCap` changes the effective cap of every gauge whose stored entry is `DEFAULT_CAP_INDICATOR` without any checkpoint:

```
function setDefaultCap(uint128 _cap) external override {
    require(hasRole(CAP_ADMIN_ROLE, msg.sender), 'NA');
    require(_cap != 0, 'ZC');
    defaultCap = _cap;
    emit DefaultCapSet({_cap: _cap});
}
```

`emissionCap(gauge)` resolves the cap at settlement time via `cap == DEFAULT_CAP_INDICATOR ? defaultCap : cap`. Because settlement is deferred until the next interaction with each gauge, a `setDefaultCap` call changes the cap that will be applied retroactively to the entire unsettled period for every affected gauge, rather than taking effect only from the next settlement forward.

#012 A Sequencer Outage Can Lead to Slashing

Informational

Version 1

Open 

In `SlipstreamStaker`, the early-unstake penalty window is counted in blocks. `unstakeToken()` becomes unpermissioned `REFUND_DELAY` after `endTime`, which is one hour. The constructor reconciles the two by assuming a minimum rate of block production, `microsPerBlock`. A sequencer outage could lead to fewer than `minStakeBlocks` blocks produced, so a position staked shortly before `endTime` could get slashed.

#013 Any Contract Is Accepted as the Reward Token of an Incentive

Informational

Version 1

Open 

`SlipstreamStaker.createIncentive()` does not validate `key.rewardToken`. It pulls the reward through `TransferHelperExtended.safeTransferFrom()`, which only requires that the address is a contract and then calls `transferFrom()` on it. A call that does not revert and returns empty data is treated as a success.

```
(bool success, bytes memory data) =
    token.call(abi.encodeWithSelector(IERC20.transferFrom.selector, from, to, value));
require(success && (data.length == 0 || abi.decode(data, (bool))), 'STF');
```

Passing `nonfungiblePositionManager` as `key.rewardToken` therefore moves the caller's position with token id `reward` into the staker, since the ERC-721 `transferFrom()` shares the ERC-20 selector and returns no data. It only requires the staker to be approved for that token id. As this is a plain transfer and not a safe transfer, `onERC721Received()` is not invoked and no `deposits[tokenId]` entry is written.

The token id is accounted as a reward amount in `totalRewardUnclaimed`, and `unstakeToken()` consumes it and credits fractions of it to `rewards[key.rewardToken][deposit.owner]`. These fractions can in theory match existing NFT IDs that are currently staked in the contract. However, the IDs cannot be paid out. `endIncentive()` and `claimReward()` transfer tokens with `TransferHelperExtended.safeTransfer()`:

```
(bool success, bytes memory data) = token.call(abi.encodeWithSelector(IERC20.transfer.selector, to, value));
require(success && (data.length == 0 || abi.decode(data, (bool))), 'ST');
```

The position manager implements no `transfer(address,uint256)` and has no fallback function, so both calls revert and the position remains locked in the staker.

The impact is confined to the incentive creator, who has to move their own position and approve the staker for it. Beyond that, NFT token ids enter the reward accounting as amounts. The resulting `totalRewardUnclaimed`, `totalRewardSlashed`, and per-user `rewards` balances have no relation to a transferable reward balance, and the resulting state is not one we can fully reason about.

#014 Claim Amount Is Remapped in Undocumented Ways

Informational

Version 1

Open 

In `SlipstreamStaker`, `_claimReward()` treats `amountRequested` as a sentinel rather than as the amount to transfer. Zero means claim the whole balance, and any value above the account's `rewards` balance is silently reduced to that balance.

```
reward = rewards[rewardToken][account];
if (amountRequested != 0 && amountRequested < reward) {
    reward = amountRequested;
}
```

Neither behavior appears in the documentation. `ISlipstreamStaker` documents `claimReward()` as transferring "`amountRequested` of accrued `rewardToken` rewards from the contract to the recipient `to`".

#015 CLPoolTape Called by Swap Hook Instead of Pool Directly

Informational

Version 1

Open 

The `afterSwap` hook invoked by the `CLPool` calls `CLPoolTape.record()` with data (fees, volumes, tick, staked liquidity) that the pool already has at the end of its swap execution.

```
bytes memory _calldata = abi.encodeWithSelector(
    ICLPoolTape.record.selector,
    _pool,
    ICLPoolTape.CLPoolTapeData({
        /// @dev Fee is always paid on the input amount.
        fee0: _swapParams.zeroForOne ? _fee : 0,
        fee1: _swapParams.zeroForOne ? 0 : _fee,
        volume0: _swapParams.zeroForOne ? uint128(_inputAmount) : uint128(_outputAmount),
        volume1: _swapParams.zeroForOne ? uint128(_outputAmount) : uint128(_inputAmount),
        // ...
        tick: _swapParams.tick,
        stakedLiquidity: _afterSwapParams.stakedLiquidity
    })
);
```

The hook receives this data only because it was forwarded via `AfterSwapParams`, so it is acting as a pass-through with no insight of its own. Moreover, the pool tape itself is not configurable per-pool, so the values stored should be identical regardless of the specific swap hook used. Currently, every new swap hook would have to include the same call to the pool tape to record the parameters provided by the pool.

#016 Draining a Tick Range Temporarily Locks a Zero-Liquidity NFT

Informational

Version 1

Open 

`CLGauge._withdraw()` calls `nft.collect()` before unstaking, which triggers `pool.burn(tickLower, tickUpper, 0)` on the NFT manager's aggregate pool position for the NFT's tick range. `Position.update()` requires `liquidity > 0` when the delta is zero:

```
if (liquidityDelta == 0) {
    require(_self.liquidity > 0, 'NP'); // disallow pokes for 0 liquidity positions
}
```

A zero-liquidity NFT contributes nothing to this aggregate. If every other NFT that shares the same `(tickLower, tickUpper)` pair has been burned or had its liquidity removed, the NFT manager's aggregate position reaches zero, and the `collect()` call in `_withdraw()` reverts with `'NP'`, blocking withdrawal. The condition resolves as soon as any other position with the same tick range adds liquidity back.

#017 Gauge Clone Is Left Uninitialized Across an External Call

Informational

Version 1

Open

In `CLGaugeFactory`, `createGauge()` deploys the gauge with `Clones.cloneDeterministic()` and records the factory state for it, then calls `createRewards()` on the voting rewards factory, and only afterwards runs `_initializeGauge()`. The gauge is therefore deployed and externally reachable in an uninitialized state for the duration of that external call. Nothing can act on it in that window, since `CLGauge.initialize()` admits only the gauge factory and only while `pool` is still the zero address, and every other entry point reads through `pool`. The `votingRewardsFactory` is also an immutable set at construction and checked against the zero address there.

#018 Incentive Top-Ups Are Emitted as Incentive Creations

Informational

Version 1

Open

Calling `SlipstreamStaker.createIncentive()` with the same incentive key a second time tops up the `totalRewardUnclaimed` of the existing incentive, as long as it is called at or before `key.startTime`. Since `_stakeToken()` admits stakes from `key.startTime` onwards, a campaign created with `key.startTime = block.timestamp` can be topped up while positions are already staked.

The function nevertheless emits `IncentiveCreated`, with `reward` set to the top-up amount instead of the incentive's total reward. This can mislead integrators, and it violates the specification, which documents `createIncentive()` as creating a new incentive program and `reward` as the amount of reward tokens to be distributed.

#019 Inconsistent NatSpec on SwapParams

Informational

Version 1

Code Corrected

`ISwapHook.SwapParams` documents `sqrtPriceX96` and `tick` as the pool's pre-swap price and tick. That holds for `beforeSwap()`, which `CLPool.swap()` fills from the cached `slot0` values, but not for `afterSwap()`, which it fills from the swap loop's final state and whose own parameter documentation describes the same struct as the final pool state. Since one struct serves both entry points, the documentation is inconsistent.

Code Corrected

The NatSpec on `SwapParams` was updated. `sqrtPriceX96` and `tick` are now documented as pre-swap values in `beforeSwap()` and post-swap values in `afterSwap()`.

#020 Interface Mismatch of IGauge

Informational

Version 1

Open

`IGauge` in this repository is not consistent with the interface of the same name in the metadex repository. The MetaDEX Slipstream declaration:

- omits the `voter()` getter

- adds a `ClaimFees` event
- types `gaugeFactory()` as `ICLGaugeFactory`, not `address`
- replaces the custom errors with `require` strings, since Solidity 0.7.6 does not support them

An integration built against the metadex interface therefore cannot decode this gauge's reverts and cannot query the `voter()` function on it.

#021 Magic Value of 0 for Reward Growth

Informational	Version 1	Open 
---------------	-----------	--

In `CLGauge._deposit()`, the call `pool.getRewardGrowthInside(tickLower, tickUpper, 0)` passes a literal `0` to signal that the current `rewardGrowthGlobalX128` should be read from state. A named constant would make the intent of that argument more explicit.

#022 Module Unstaked Fees Can Reach 100%, Twice the Factory Cap

Informational	Version 1	Open 
---------------	-----------	--

`CLFactory.setDefaultUnstakedFee` enforces `_fee <= 500_000` (50%).
`CustomUnstakedFeeModule.setCustomFee` enforces the same `MAX_FEE = 500_000`. However, the acceptance guard in `getUnstakedFee` is twice as permissive:

```
try IFeeModule(unstakedFeeModule).getFee(pool) returns (uint24 fee) {
    if (fee <= 1_000_000) return fee;
} catch {}
```

A buggy or malicious fee module can return any value up to `1_000_000` (100%), and the factory will forward it to the pool without clamping. At a 100% unstaked fee, every swap token owed to unstaked LPs is redirected to staked LPs, leaving unstaked liquidity providers with zero fee income.

#023 Package.json Files Entry Omits Contracts/periphery/libraries

Informational	Version 1	Open 
---------------	-----------	--

The root `package.json` `files` array contains the entry `"contracts/peripherylibraries"` (missing the path separator) instead of `"contracts/periphery/libraries"`:

```
"files": [
  "contracts/periphery/base",
  "contracts/periphery/interfaces",
  "contracts/peripherylibraries",
  ...
]
```

npm matches paths literally, so `contracts/periphery/libraries/` is not included in the published package. Consumers who install the package and import from paths that are in-scope (`contracts/periphery/interfaces`, `contracts/periphery/base`) but which transitively depend on `contracts/periphery/libraries` will receive broken imports.

#024 Read-Only Re-Entrancy in NonfungiblePositionManager's Collect

Informational

Version 1

Open

`NonfungiblePositionManager.collect()` reduces `tokensOwed0` and `tokensOwed1` only after `CLPool.collect()` has already transferred the tokens out. If the token hands control to the recipient during that transfer, the view function `positions()` still reports amounts that have already been paid out. Re-entrant writes that could exploit these stale values have to pass back through `CLPool`, where the `lock` modifier rejects them. However, integrations consuming the views are not covered by that guard and could over-value a position.

#025 Redundant Account Argument in withdrawTokenFrom

Informational

Version 1

Open

In `SlipstreamStaker`, `withdrawTokenFrom()` takes an `account` argument and then requires it to equal `deposits[tokenId].owner`, a value it has already loaded into memory. The argument can therefore hold nothing else. The only difference it makes is the non-zero check on it, which, however, is equivalent to requiring the deposit to be initialized as enforced by the code. The argument is therefore redundant.

#026 Referral Parameters Are Decoded Without a Length Check

Informational

Version 1

Open

In `CLGaugeFactory`, `_applyReferralParams()` accepts the factory data whenever it is non-empty and its first byte is `PARAMS_REFERRAL`, and `_decodeReferral()` then decodes the remainder with `abi.decode(_params[1:], (address, uint256))` without checking how long it is. If data is longer than the one flag byte plus two words the tail is ignored, while data shorter than that reverts inside the decode. The equivalent check in the V2 GaugeFactory checks that the size is exactly `_REFERRAL_PARAMS_LENGTH` (i.e. 65).

#027 Reward Is Not Bounded by the Incentive's Unclaimed Balance

Informational

Version 1

Open

`SlipstreamStaker.unstakeToken()` subtracts `reward` from `totalRewardUnclaimed` without bounding it by the balance of the specific incentive being claimed. The relationship is indirectly enforced, since the value of `rewards` computed by `RewardMath.computeRewardAmount()` is lower than the total unclaimed balance, as long as `secondsInsideX128` is at or below `totalSecondsUnclaimedX128`. However, as the staker is compiled with Solidity 0.7.6, a violation could underflow `totalRewardUnclaimed` with potentially catastrophic effects.

#028 SlipstreamStaker Constructor Misses Zero Address Checks

Informational

Version 1

Open

In `SlipstreamStaker`, the constructor rejects a zero `owner` but stores `factory`, `nonfungiblePositionManager` and `leafVoter` unchecked, and all three are immutable so a zero value cannot be corrected after deployment.

#029 withdrawToken Accepts the Zero Address as Recipient

Informational

Version 1

Open 

In `SlipstreamStaker`, `withdrawToken()` rejects the staker itself as `to` but not the zero address, while `transferDeposit()` and `withdrawTokenFrom()` both reject it. A zero `to` reverts only because the position manager derives from OpenZeppelin's `ERC721`, whose `safeTransferFrom()` refuses a zero recipient.

8 Open Questions

Here, we list open questions that came up during the assessment and that we would like to clarify to ensure that no important information is missing.

OQ1 Should createIncentive Verify the Pool Is Factory-Deployed?

Open Question	Version 1
---------------	-----------

In `SlipstreamStaker`, `createIncentive()` accepts any `key.pool` without checking that it is a factory-deployed pool via `factory.isPool()`. Function `_stakeToken()` compares `key.pool` against the pool derived from the NPM token id, so an incentive campaign referencing a pool that is not deployed by the factory cannot be staked on, and its reward stays in the staker until `endIncentive()` returns it to `key.refundee` after `endTime + REFUND_DELAY`.

OQ2 Why Do Flash Loan Fees Accrue Only to in-Range Liquidity?

Open Question	Version 1
---------------	-----------

`CLPool.flash()` splits flashloan fees between in-range `liquidity` and `stakedLiquidity`. Minting a narrow position spanning the current tick is permissionless, so a borrower can supply the in-range liquidity itself, take the loan, and collect the unstaked share back through `feeGrowthGlobal`.



9 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.